

Start coding or generate with AI.

⚡ WARNING:tensorflow:From <ipython-input-1-63f2763fc1eb>:10: experimental_run_functions_eagerly is deprecated. Instructions for updating:
Use `tf.config.run\_functions\_eagerly` instead of the experimental version.

```
! pip install kaggle
```

⚡ Requirement already satisfied: kaggle in /usr/local/lib/python3.11/dist-packages (1.6.17)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.11/dist-packages (from kaggle) (1.16.0)
Requirement already satisfied: certifi>=2023.7.22 in /usr/local/lib/python3.11/dist-packages (from kaggle) (2024.7.4)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.8.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.32.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.11/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.11/dist-packages (from kaggle) (2.2.3)
Requirement already satisfied: bleach in /usr/local/lib/python3.11/dist-packages (from kaggle) (6.1.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.11/dist-packages (from kaggle) (0.11)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.11/dist-packages (from kaggle) (1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from kaggle) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from kaggle) (3.10)

```
! mkdir ~/.kaggle
```

```
! cp kaggle.json ~/.kaggle/
```

```
! chmod 600 ~/.kaggle/kaggle.json
```

```
! kaggle datasets download preetviradiya/brian-tumor-dataset
```

⚡ Dataset URL: <https://www.kaggle.com/datasets/preetviradiya/brian-tumor-dataset>
License(s): GPL-2.0
Downloading brian-tumor-dataset.zip to /content
91% 98.0M/107M [00:01<00:00, 73.9MB/s]
100% 107M/107M [00:01<00:00, 75.3MB/s]

```
from google.colab import drive
drive.mount('/content/gdrive')
```

⚡ Mounted at /content/gdrive

```
mkdir /content/dataset/
```

```
! unzip "/content/brian-tumor-dataset.zip" -d /content/dataset
```



```

initializing: /content/dataset/metadata.csv
inflating: /content/dataset/metadata_ngh_only.csv

```

```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import keras
import tensorflow as tf
from keras.preprocessing.image import load_img
from tensorflow.keras.preprocessing.image import ImageDataGenerator
tf.config.experimental_run_functions_eagerly(True)

img_dir = '/content/dataset/Brain Tumor Data Set/Brain Tumor Data Set'

BATCH_SIZE = 64
IMAGE_SIZE = 150
input_shape = (150,150,1)

```

```

data_gen = ImageDataGenerator(rescale=1./255,
                              validation_split = 0.2)

```

```

train_gen = data_gen.flow_from_directory(img_dir,
                                         target_size=(IMAGE_SIZE,IMAGE_SIZE),
                                         batch_size = BATCH_SIZE,
                                         color_mode= "grayscale",
                                         shuffle=True,
                                         class_mode = "binary",
                                         subset = "training")

```

➡ Found 3681 images belonging to 2 classes.

```

val_gen = data_gen.flow_from_directory(img_dir,
                                       target_size=(IMAGE_SIZE,IMAGE_SIZE),
                                       batch_size = BATCH_SIZE,
                                       color_mode= "grayscale",
                                       shuffle=False,
                                       class_mode = "binary",
                                       subset = "validation")

```

➡ Found 919 images belonging to 2 classes.

```

labels = train_gen.class_indices
classes = list(labels.keys())

print(classes)

```

→ ['Brain Tumor', 'Healthy']

✓ Making the model

```
from keras.models import Sequential
from keras.layers import Dense, Conv2D, MaxPooling2D, BatchNormalization, Flatten, Dropout
```

```
model = Sequential()
model.add(keras.layers.InputLayer(input_shape=(150, 150, 1)))
model.add(Conv2D(16,(3,3), activation="relu"))
model.add(MaxPooling2D((2,2)))
model.add(Conv2D(32,(3,3), activation="relu"))
model.add(MaxPooling2D((2,2)))
model.add(Flatten())
model.add(Dense(512, activation="relu"))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))
```

→ /usr/local/lib/python3.11/dist-packages/keras/src/layers/core/input_layer.py:27: UserWarning



```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 16)	160
max_pooling2d (MaxPooling2D)	(None, 74, 74, 16)	0
conv2d_1 (Conv2D)	(None, 72, 72, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 32)	0
flatten (Flatten)	(None, 41472)	0
dense (Dense)	(None, 512)	21,234,176
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 1)	513

Total params: 21,239,489 (81.02 MB)
 Trainable params: 21,239,489 (81.02 MB)
 Non-trainable params: 0 (0.00 B)

with tf.device('/device:GPU:0'):

history = model.fit(train_gen, verbose=1, epochs=5, validation_data=val_gen, steps_per_epoch

```

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:100:
self._warn_if_super_not_called()
Epoch 1/5
/usr/local/lib/python3.11/dist-packages/tensorflow/python/data/ops/structured_function_wrapper.py:100:
warnings.warn(
57/57 ————— 147s 3s/step - accuracy: 0.6048 - loss: 1.1197 - val_accuracy: 0.8125
Epoch 2/5
1/57 ————— 1:51 2s/step - accuracy: 0.8125 - loss: 0.4256/usr/local/lib/python3.11/dist-packages/tensorflow/python/data/ops/structured_function_wrapper.py:100:
self._interrupted_warning()
57/57 ————— 22s 354ms/step - accuracy: 0.8125 - loss: 0.4256 - val_accuracy: 0.8125
Epoch 3/5
57/57 ————— 176s 3s/step - accuracy: 0.8722 - loss: 0.3168 - val_accuracy: 0.8722
Epoch 4/5
57/57 ————— 20s 314ms/step - accuracy: 0.9062 - loss: 0.3287 - val_accuracy: 0.9062
Epoch 5/5
57/57 ————— 203s 3s/step - accuracy: 0.9423 - loss: 0.1555 - val_accuracy: 0.9423

```

```

#train_loss, train_acc = model.evaluate(train_gen, steps_per_epoch = 3681//64, validation_steps=3681//64)
test_loss, test_acc = model.evaluate(val_gen)

```

```

15/15 ————— 25s 2s/step - accuracy: 0.8681 - loss: 0.3093

```

```

loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(1, len(loss) + 1)

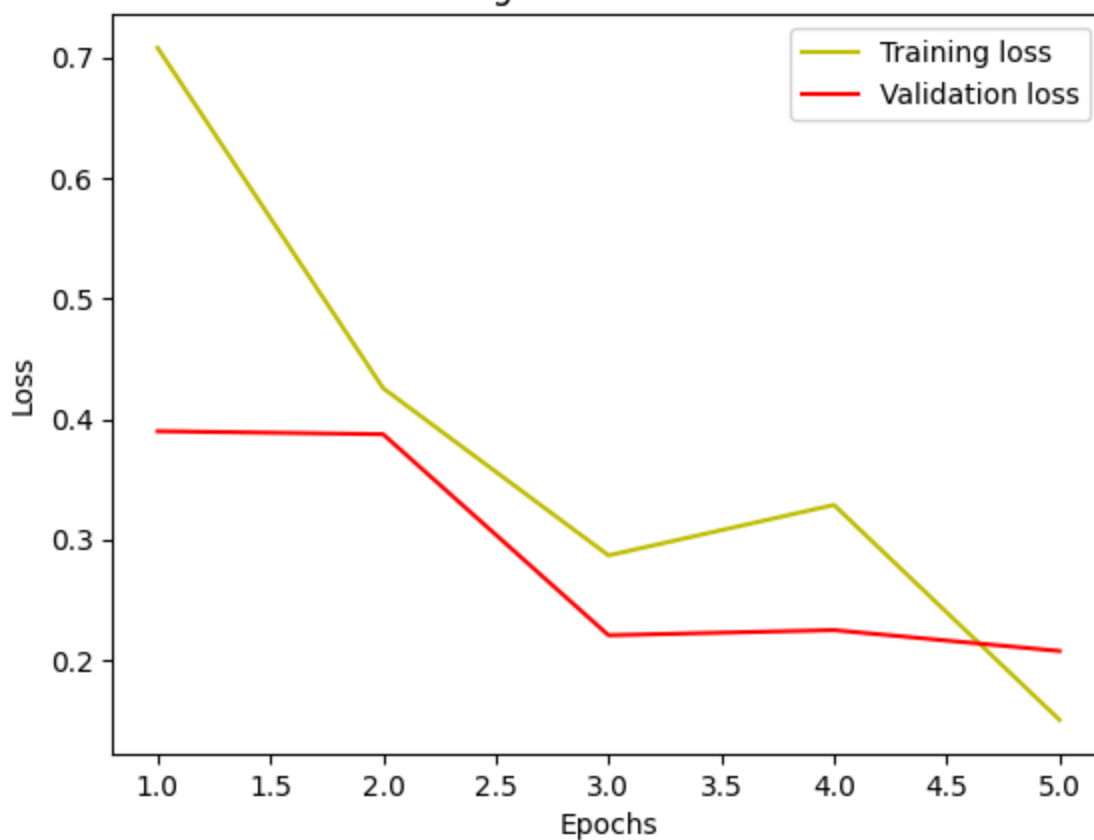
```

```
plt.plot(epochs, loss, 'y', label='Training loss')
plt.plot(epochs, val_loss, 'r', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

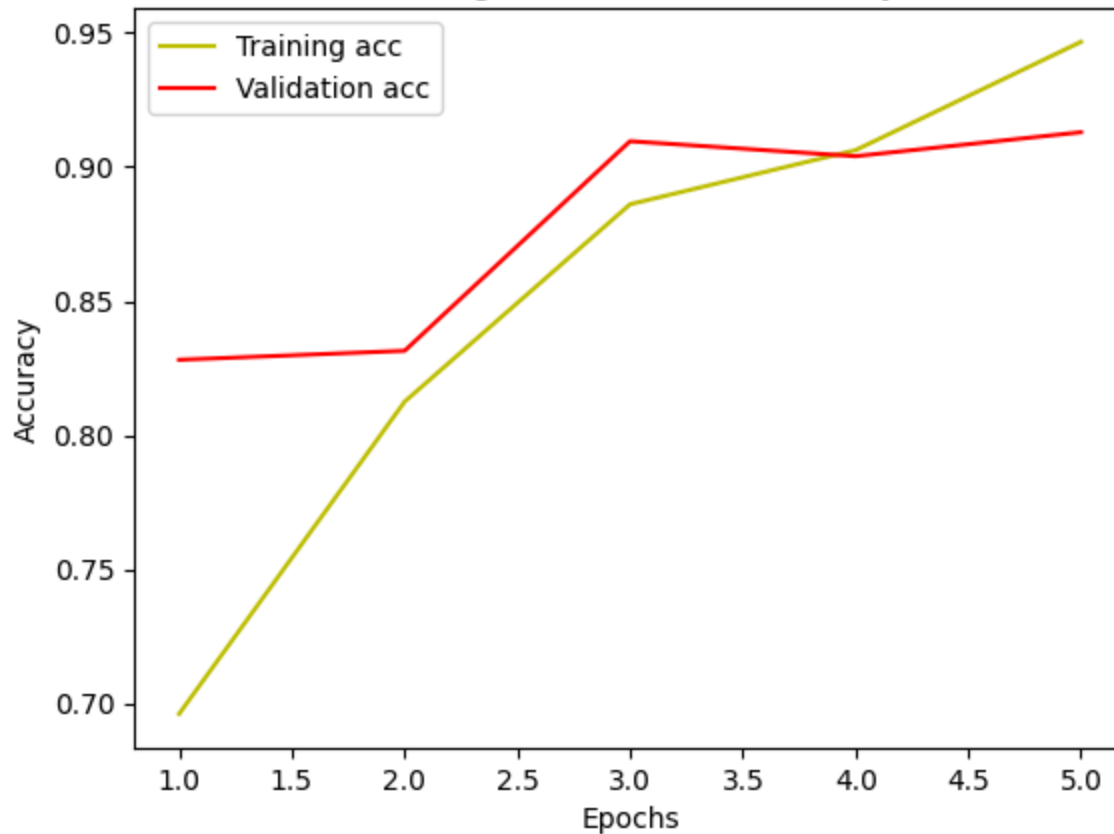
```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
plt.plot(epochs, acc, 'y', label='Training acc')
plt.plot(epochs, val_acc, 'r', label='Validation acc')
plt.title('Training and validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



Training and validation loss



Training and validation accuracy



```
model.save('model.h5')
```



WARNING:absl:You are saving your model as an HDF5 file via ``model.save()`` or ``keras.savi`

